Arquitectura de Software

Arquitectura Hexagonal

Gestor de Citas Médicas

Vue · Go · GraphQL · PostgreSQL
Presentación 01 · 2026-04-13

Erick Salazar · Felipe Garrido · Augusto Pedicino

Contenido de la presentación

01 **Definición del estilo**
Arquitectura Hexagonal — qué es y qué no es

02 **Stack tecnológico**
Go, Vue, GraphQL, PostgreSQL

03 **Análisis arquitectónico**
Matrices de calidad, principios, tácticas y mercado

04 **Diseño C4**
HLD, Contexto, Contenedores, Dinámico, Despliegue

05 **Implementación**
Estructura real del proyecto y Docker

06 **Lecciones aprendidas**
Reflexiones del equipo

¿Qué es la Arquitectura Hexagonal?

También conocida como Ports & Adapters — Alistair Cockburn, años 2000

Definición

- Aísla la lógica de negocio del resto del sistema.
- Dominio (Core), Puertos (Ports) y Adaptadores (Adapters).
- El núcleo no depende de tecnologías externas.

Clasificación

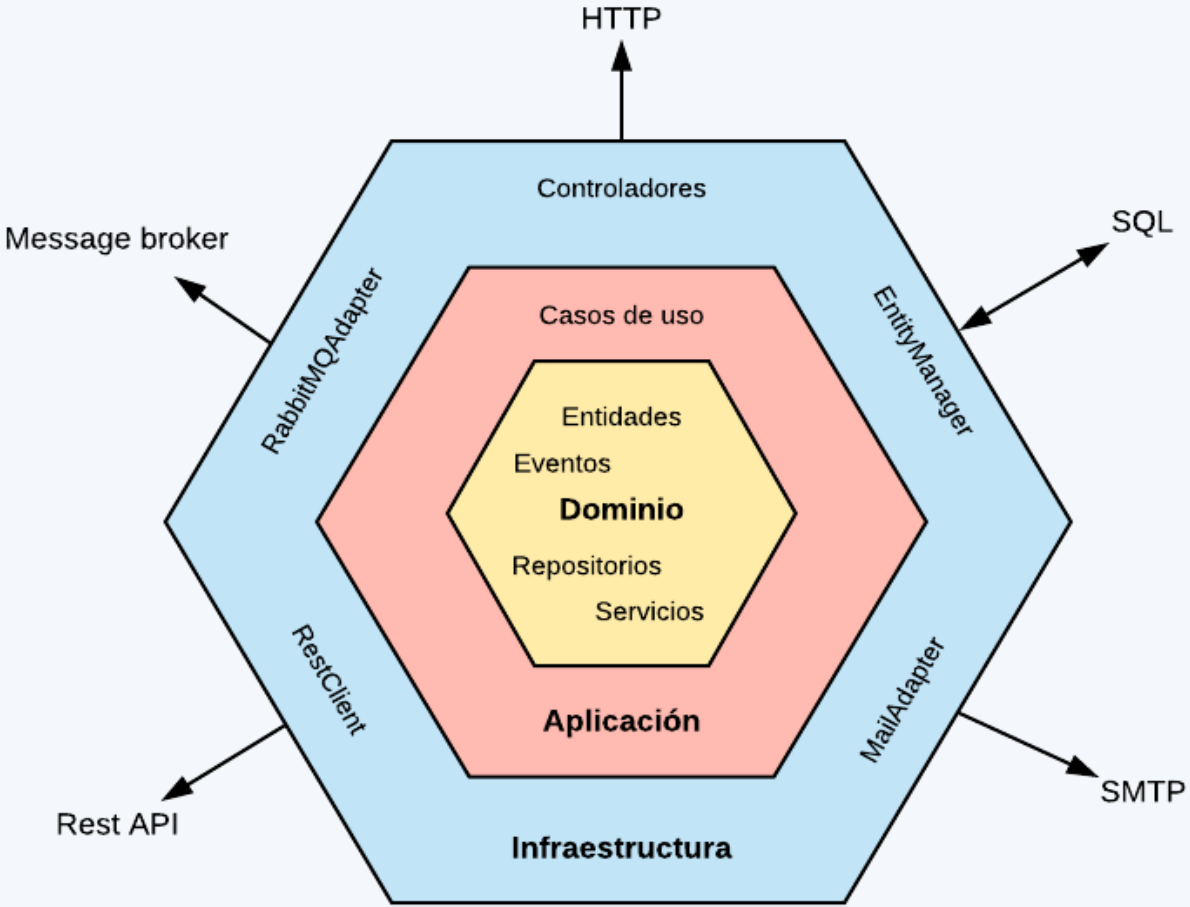
- Estilo arquitectónico estructural.
- Arquitectura centrada en el dominio.
- Relacionada con Clean Architecture, Onion y DDD.

¿Qué NO es?

- No es un framework ni una librería.
- No es solo separación de carpetas.
- No es arquitectura por capas tradicional.
- No es obligatoria para sistemas pequeños.

Historia

- Propuesta por Alistair Cockburn en 2000.
- Respuesta al acoplamiento fuerte y dificultad de testeo.
- Se integró con DDD y Clean Architecture.
- Hoy adoptada en microservicios y APIs modernas.


HTTP
Controladores
Message broker
SQL
RabbitMQAdapter
EntityManager
Casos de uso
Entidades
Eventos
Dominio
Repositorios
Servicios
RestClient
MailAdapter
Aplicación
Rest API
SMTP
Infraestructura

Características, ventajas y desventajas

Arquitectura Hexagonal

Características

- Separación de responsabilidades.
- Inversión de dependencias.
- Adaptabilidad tecnológica.
- Alta testabilidad.
- Modularidad.
- Desarrollo desacoplado.

Ventajas

- Alta mantenibilidad.
- Fácil testeo sin infraestructura.
- Flexibilidad tecnológica.
- Código limpio y organizado.
- Cambio de DB sin tocar el dominio.
- Escalabilidad progresiva.

Desventajas

- Mayor complejidad inicial.
- Curva de aprendizaje alta.
- Más boilerplate (interfaces, adapters).
- Overhead en proyectos pequeños.
- Debugging más complejo.
- Requiere disciplina de equipo.

Stack tecnológico

Vue · Go/Echo · GraphQL · PostgreSQL

Frontend

Vue.js + Nuxt 4

SPA reactiva. Apollo Client consume GraphQL. Componentes por dominio: Pacientes, Doctores, Citas.

Backend

Go 1.22

Alto rendimiento y concurrencia nativa. gqlgen genera el servidor GraphQL. Arquitectura Hexagonal sobre paquetes Go.

Protocolo

GraphQL

Queries y Mutations tipadas. Schema único como contrato. Evita over-fetching en el frontend.

Base de datos

PostgreSQL 16

ACID, integridad relacional. GORM como ORM. Tablas: pacientes, doctores, citas.

Patrones aplicados al sistema

Cuándo y por qué en el Gestor de Citas

Repository

Abstrae el acceso a PostgreSQL. CitaRepository, DoctorRepository, PacienteRepository como puertos de salida.

Dependency Injection

Los casos de uso reciben sus repos por constructor. DI manual en Go sin frameworks adicionales.

Service Layer

CitaService, DoctorService, PacienteService orquestan los casos de uso sin conocer infraestructura.

Adapter (GraphQL)

Los resolvers de schema.resolvers.go actúan como adaptadores de entrada hacia el dominio.

DTO / Model Gen

gqlgen genera models_gen.go separando el modelo GraphQL de las entidades de dominio.

Error tipado

Errores de dominio (ErrCitaNoDisponible) separados de errores de infraestructura.

Atributos de calidad ISO/IEC 25000

Cómo la Arquitectura Hexagonal soporta o limita cada atributo

Atributo	Soporte	Descripción
Adecuación Funcional	Alto	Casos de uso aislados y completos en el dominio.
Eficiencia de desempeño	Medio	Go muy rápido; riesgo de N+1 en GraphQL sin DataLoader.
Compatibilidad	Alto	Puertos como contratos; GraphQL agnóstico al cliente.
Usabilidad	Medio	Dominio limpio mejora UX; stack añade curva de aprendizaje.
Fiabilidad	Medio	Errores tipados; sin K8s la disponibilidad es limitada.
Seguridad	Medio	Middleware JWT como adaptador; GraphQL expone introspección.
Mantenibilidad	Alto	Única razón de cambio por capa; interfaces testeables con mocks.
Portabilidad	Alto	Cambio de DB = solo reescribir el adaptador. Docker encapsula todo.

Principios de diseño vs Arquitectura Hexagonal

SOLID · KISS · DRY · YAGNI · PoLA · Demeter · STUPID · Composición

Principio	Cumple	Relación con la hexagonal
SRP	Sí	Cada capa tiene una única razón de cambio.
OCP	Sí	Nuevos adaptadores sin modificar el dominio.
LSP	Sí	CitaRepositoryMock sustituye al real sin cambios.
ISP	Sí	Puertos pequeños y específicos por caso de uso.
DIP	Sí	El dominio define interfaces; la infra las implementa.
KISS	Parcial	Múltiples capas pueden ser excesivas en sistemas pequeños.
DRY	Sí	Validaciones viven una sola vez en el dominio.
YAGNI	Parcial	Riesgo de crear puertos que nunca se usan.
PoLA	Sí	Estructura predecible: domain, application, adapters.
Demeter	Sí	Casos de uso solo llaman a su puerto inmediato.
STUPID	Evitado	No hay Singleton, Tight Coupling ni God Object.
Composición	Sí	Go usa interfaces y structs; sin herencia clásica.

Decisiones arquitectónicas (ADR)

Tácticas aplicadas al stack y al estilo

ADR-001

Repository Pattern

Se usa database/sql + GORM sobre ORM puro para control del SQL.

ADR-002

GraphQL vs REST

GraphQL evita over-fetching en consultas anidadas (cita + paciente + doctor).

ADR-003

DI Manual

Sin Wire ni Uber Fx. Para 3 entidades, la DI por constructor es suficiente.

ADR-004

Validación en dominio

Validaciones en entidades Go, no en resolvers ni en constrains de la BD.

ADR-005

Error handling tipado

Errores de dominio separados de errores de infraestructura en Go.

ADR-006

Docker Compose

DATABASE_URL por variable de entorno. Sin hardcoding.

ADR-007

DataLoader (pendiente)

Mitigado con JOINs en GORM Preload. Se implementa si hay N+1 real.

ADR-008

CQRS ligero (opcional)

Query vs Mutation en GraphQL. Puertos separados si las necesidades divergen.

Mercado laboral

Posicionamiento del stack Vue · Go · GraphQL · PostgreSQL — 2024/2025

	Junior	Senior	
Go (Golang) *Creciente*	$4M–$6M COP	$10M–$20M COP	*Top 10 TIOBE · #5 lenguaje deseado SO Survey 2024*
Vue.js *Estable*	$2.5M–$4M COP	$6M–$12M COP	*3er framework frontend global · Fuerte en LATAM*
GraphQL *Creciente*	+10-20% vs REST	+10-20% vs REST	*GitHub, Shopify, Facebook · Diferenciador en perfiles*
PostgreSQL *Dominante*	Requisito base	+$1M–$3M COP avanzado	*1er DBMS SO Survey 4 años consecutivos*

Diagrama de alto Nivel (HLD)

Gestor de Citas Médicas

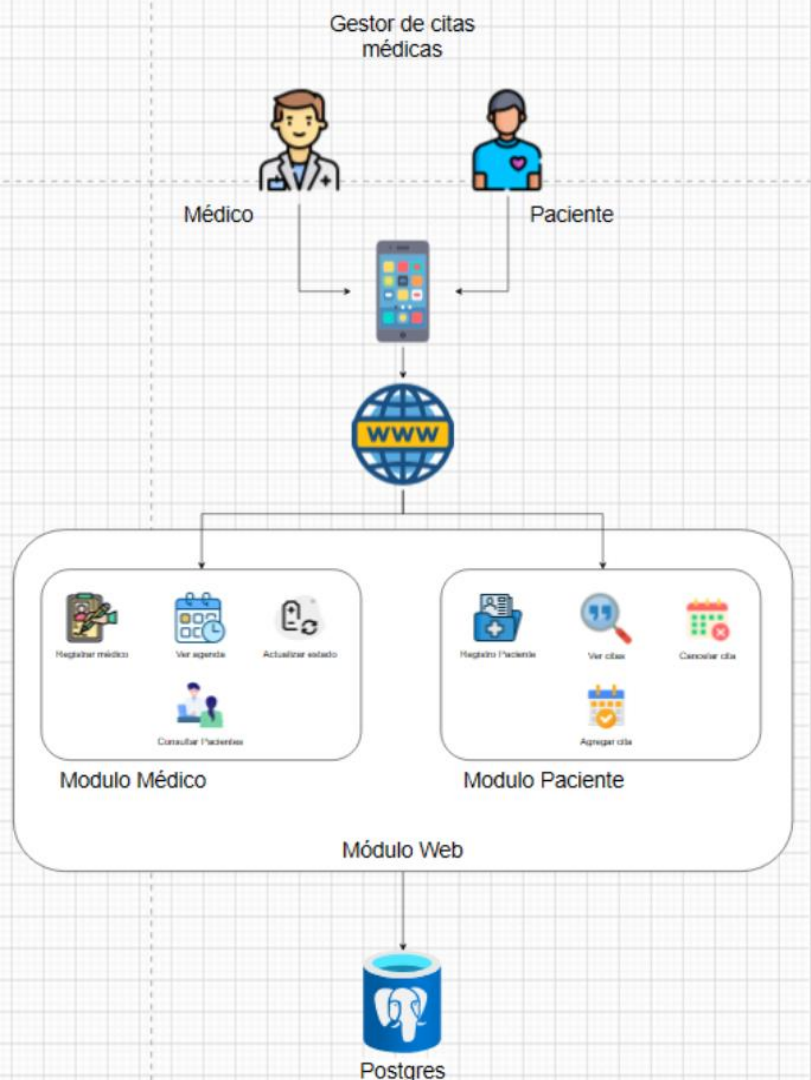

Gestor de citas médicas
Médico
Paciente
WWW
Registrar médico
Ver agenda
Actualizar estado
Consultar Pacientes
Modulo Médico
Registro Paciente
Ver citas
Cancelar cita
Agregar cita
Modulo Paciente
Módulo Web
Postgres

Diagrama de Contexto

Gestor de Citas Médicas

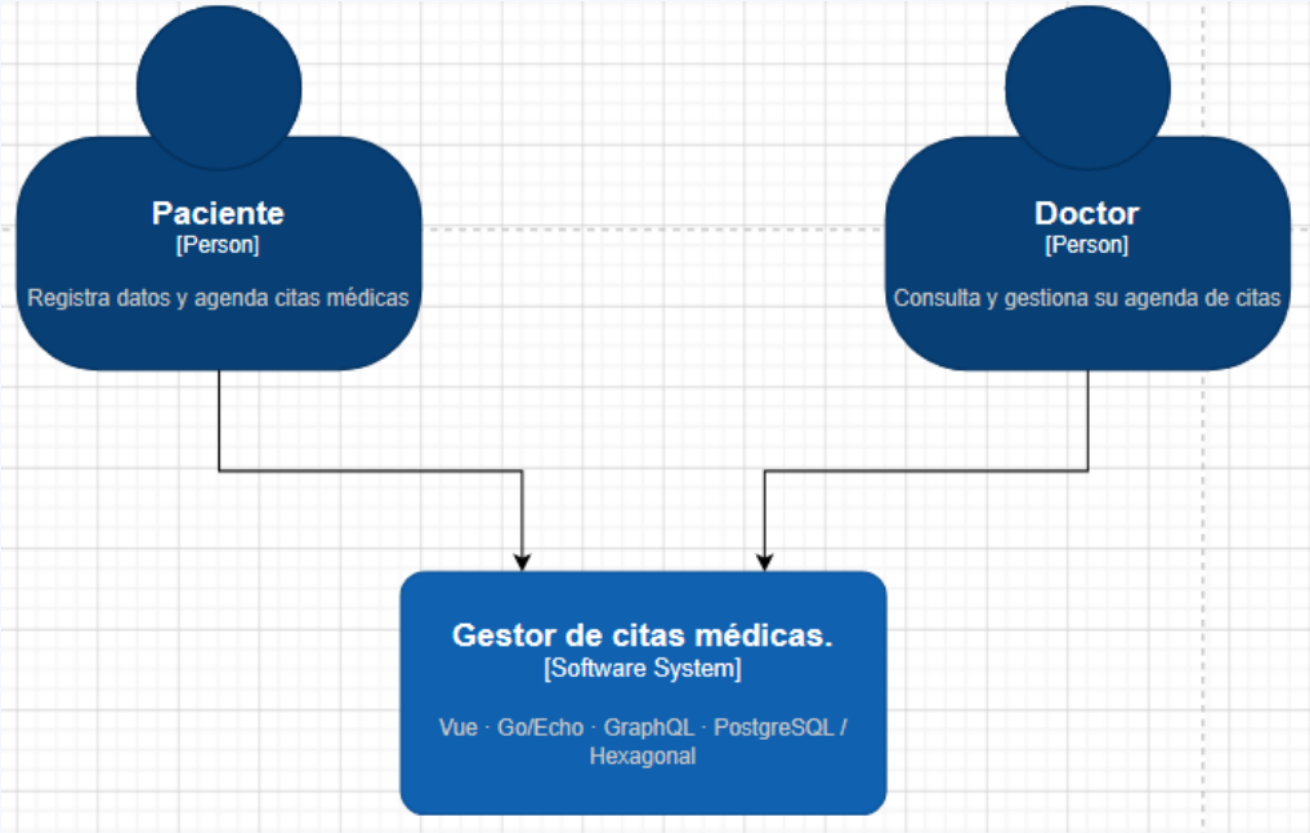

Paciente
[Person]
Registra datos y agenda citas médicas
Doctor
[Person]
Consulta y gestiona su agenda de citas
Gestor de citas médicas.
[Software System]
Vue · Go/Echo · GraphQL · PostgreSQL / Hexagonal

Diagrama de Contenedores

Gestor de Citas Médicas

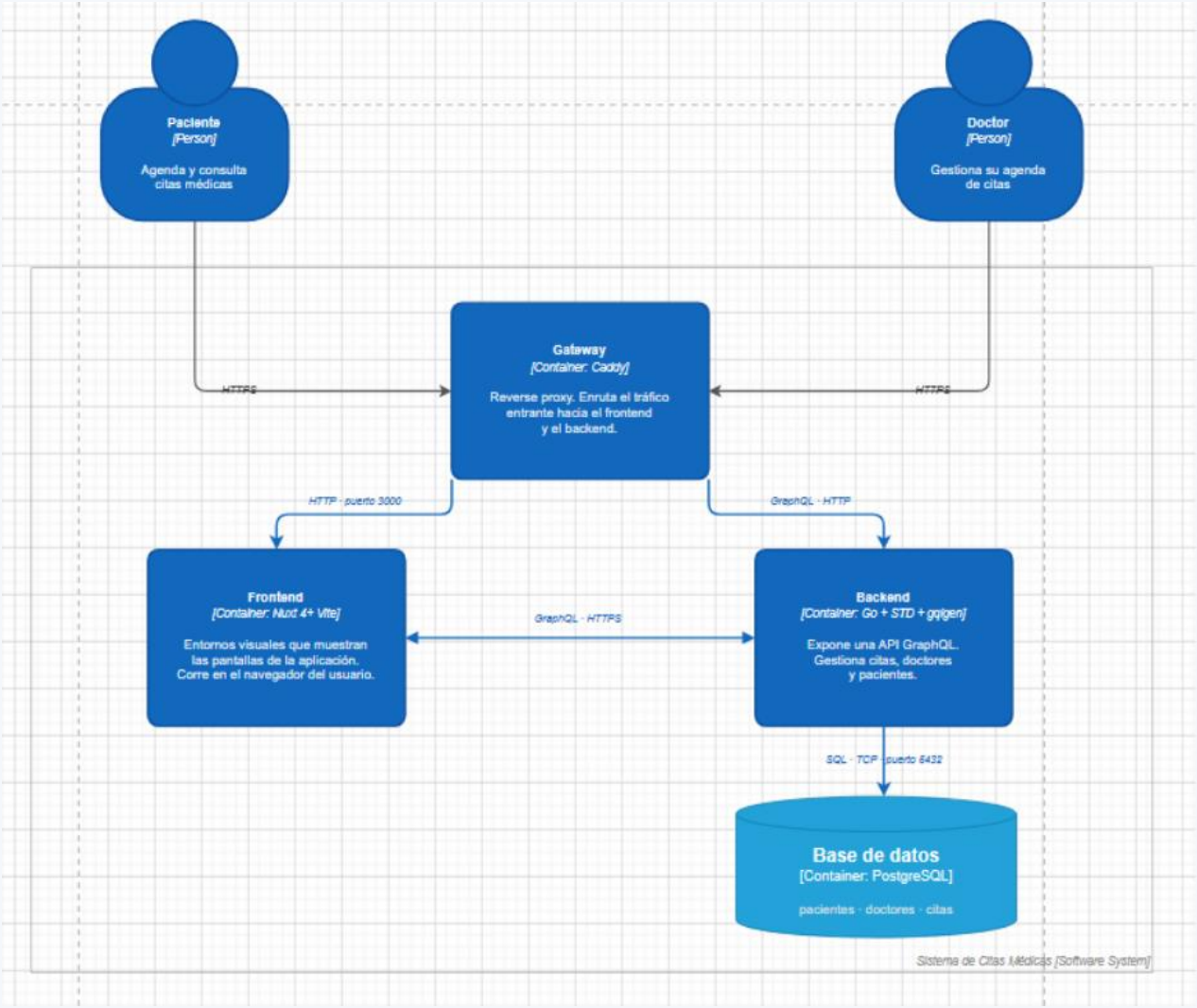
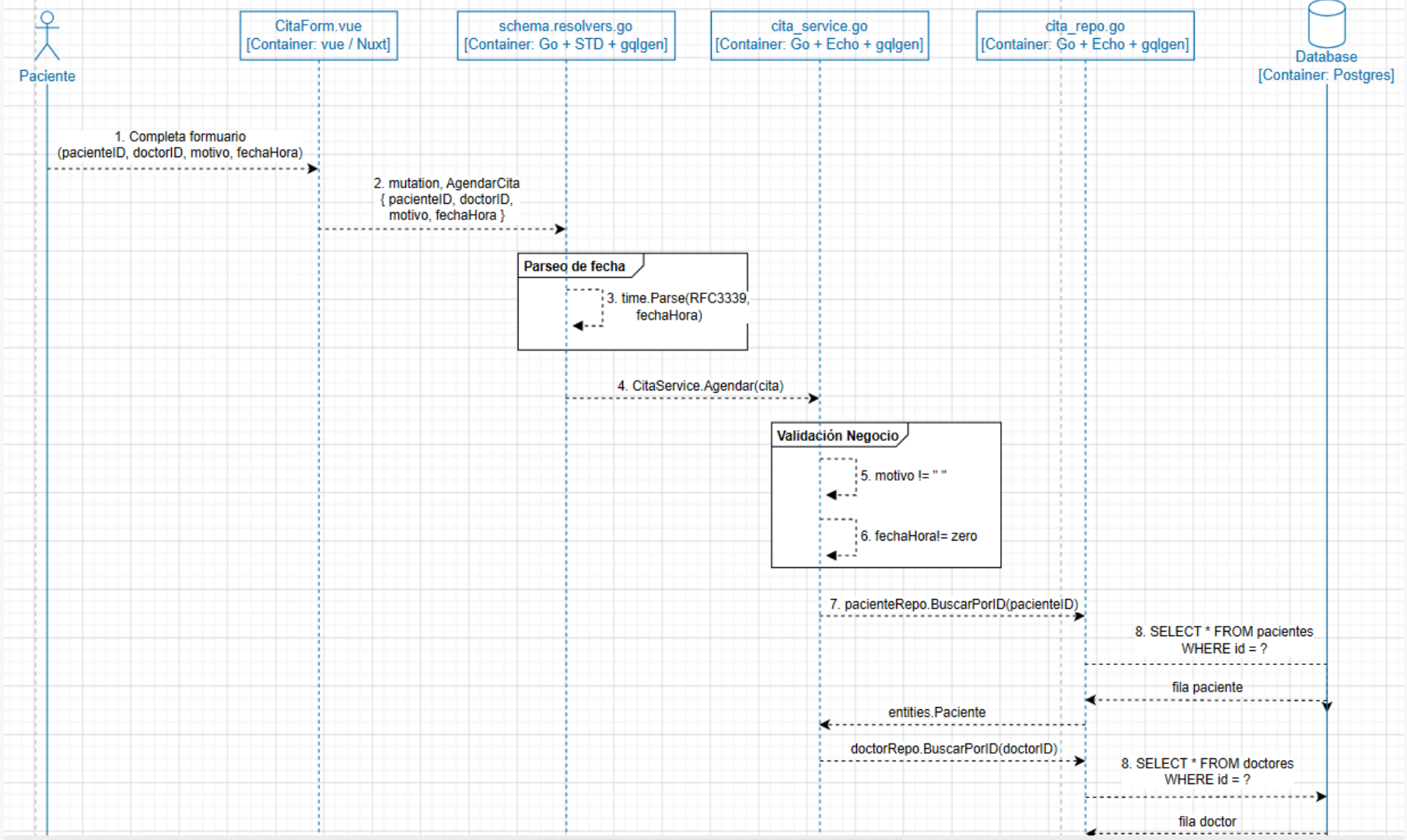

Paciente
[Person]
Agenda y consulta citas médicas
Doctor
[Person]
Gestiona su agenda de citas
HTTPS
HTTPS
Gateway
[Container: Caddy]
Reverse proxy. Enruta el tráfico entrante hacia el frontend y el backend.
HTTP · puerto 3000
GraphQL · HTTP
Frontend
[Container: Nuxt 4+ Vite]
Entornos visuales que muestran las pantallas de la aplicación. Corre en el navegador del usuario.
GraphQL · HTTPS
Backend
[Container: Go + STD + gqlgen]
Expone una API GraphQL. Gestiona citas, doctores y pacientes.
SQL · TCP · puerto 5432
Base de datos
[Container: PostgreSQL]
pacientes · doctores · citas
Sistema de Citas Médicas [Software System]

Diagrama Dinámico

Gestor de Citas Médicas


Paciente
CitaForm.vue
[Container: vue / Nuxt]
schema.resolvers.go
[Container: Go + STD + gqlgen]
cita_service.go
[Container: Go + Echo + gqlgen]
cita_repo.go
[Container: Go + Echo + gqlgen]
Database
[Container: Postgres]
1. Completa formuario
(pacienteID, doctorID, motivo, fechaHora)
2. mutation, AgendarCita
{ pacienteID, doctorID,
motivo, fechaHora }
Parseo de fecha
3. time.Parse(RFC3339,
fechaHora)
4. CitaService.Agendar(cita)
Validación Negocio
5. motivo != " "
6. fechaHora!= zero
7. pacienteRepo.BuscarPorID(pacienteID)
8. SELECT * FROM pacientes
WHERE id = ?
fila paciente
entities.Paciente
doctorRepo.BuscarPorID(doctorID)
8. SELECT * FROM doctores
WHERE id = ?
fila doctor

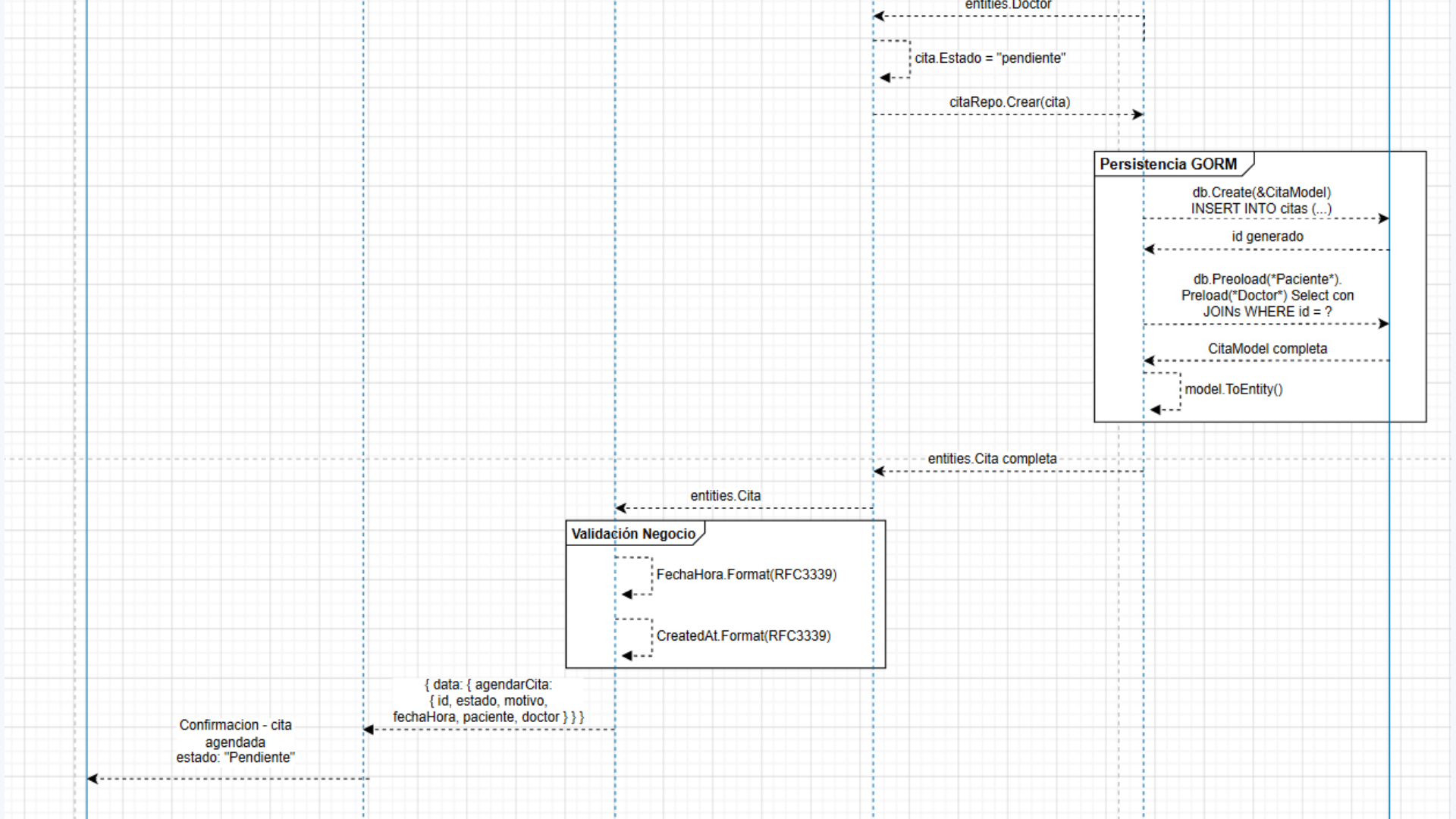
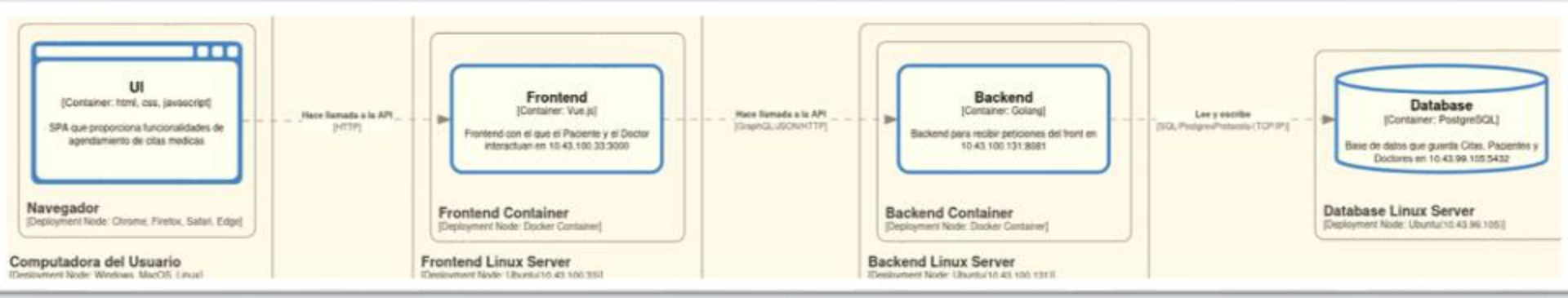

entities.Doctor
cita.Estado = "pendiente"
citaRepo.Crear(cita)
Persistencia GORM
db.Create(&CitaModel)
INSERT INTO citas (...)
id generado
db.Preload(*Paciente*).
Preload(*Doctor*) Select con
JOINs WHERE id = ?
CitaModel completa
model.ToEntity()
entities.Cita completa
entities.Cita
Validación Negocio
FechaHora.Format(RFC3339)
CreatedAt.Format(RFC3339)
{ data: { agendarCita:
{ id, estado, motivo,
fechaHora, paciente, doctor } } }
Confirmacion - cita
agendada
estado: "Pendiente"

Diagrama Despliegue

Gestor de Citas Médicas


UI
[Container: html, css, javascript]
SPA que proporciona funcionalidades de agendamiento de citas medicas
Navegador
[Deployment Node: Chrome, Firefox, Safari, Edge]
Computadora del Usuario
[Deployment Node: Windows, MacOS, Linux]
Hace llamada a la API
[HTTP]
Frontend
[Container: Vue.js]
Frontend con el que el Paciente y el Doctor interactuan en 10.43.100.33:3000
Frontend Container
[Deployment Node: Docker Container]
Frontend Linux Server
[Deployment Node: Ubuntu/10.43.100.33]
Hace llamada a la API
[GraphQL/JSON/HTTP]
Backend
[Container: Golang]
Backend para recibir peticiones del front en 10.43.100.131:8081
Backend Container
[Deployment Node: Docker Container]
Backend Linux Server
[Deployment Node: Ubuntu/10.43.100.131]
Lee y escribe
[SQL/PostgreProtocolo/TCP/IP]
Database
[Container: PostgreSQL]
Base de datos que guarda Citas, Pacientes y Doctores en 10.43.99.105:5432
Database Linux Server
[Deployment Node: Ubuntu/10.43.99.105]

Implementación

Estructura real del proyecto · Repo Git · Docker

Estructura del backend (Go)

- domain/entities/ — Cita, Doctor, Paciente
- domain/ports/ — interfaces de repositorio
- application/ — cita_service, doctor_service…
- adapters/postgresql/ — implementaciones GORM
- transport/graphql/ — resolvers, schema, generated
- infrastructure/db/ — conexión PostgreSQL
- main.go — wiring de dependencias

Estructura del frontend (Nuxt)

- app/pages/ — citas, doctores, pacientes
- app/components/ — formularios y cards
- app/graphql/ — schema.graphql + queries

Despliegue

- Docker Compose: frontend + backend + db
- DATABASE_URL por variable de entorno
- Multi-stage build en Go para imagen ligera
- Volumen Docker para persistencia PostgreSQL
- Repositorio Git público con TAG y Release

Resultado

Lecciones aprendidas

Reflexiones del equipo

01

La hexagonal vale su costo

El boilerplate inicial (puertos, adaptadores, casos de uso) se justifica desde el primer cambio de tecnología. Cambiar de database/sql a GORM no tocó el dominio.

02

GraphQL y N+1

Los Preloads de GORM mitigan el problema sin DataLoader. Pero es un riesgo real que hay que monitorear desde el diseño del schema.

03

Go + interfaces = hexagonal natural

Go no tiene herencia. Las interfaces y la composición de structs encajan perfectamente con los puertos y adaptadores sin frameworks adicionales.

04

Docker Compose simplifica el despliegue

Tener los tres contenedores declarados permite reproducir el entorno en cualquier VM de los integrantes con un solo comando.

05

gqlgen genera código facilmente

El archivo generated.go y models_gen.go eliminan trabajo manual y mantienen el schema como fuente de verdad entre frontend y backend.

06

El schema GraphQL es el contrato

Cualquier cambio en schema.graphqls se propaga automáticamente. Mantenerlo sincronizado entre Vue y Go fue clave para evitar bugs de integración.

Gracias

Vue · Go · GraphQL · PostgreSQL

Arquitectura Hexagonal — Gestor de Citas Médicas

Pontificia Universidad Javeriana · 2026

¿Preguntas?